GReinSS

Generative Modeling of Discrete Latent Structures via Dynamic Policy Gradients

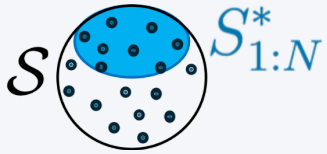
Mohammed El-Kebir
University of Illinois Urbana-Champaign

NCI Spring School on Algorithmic Cancer Biology — Tutorial

Ivanovic et al., ICML 2026

A recurring statistical inference problem in computational biology

States $S_{1:N} \sim \Pr^*(\mathcal{S})$



Measurements $X_{1:N}$ generated from $S_{1:N}$

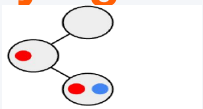
X_i :

$\mathcal{S}$ is typically **large and combinatorial** — graphs, strings, sets, …

S_i^* :

Rather than directly observing the latent **state** S we care about, we observe some indirect **measurement** X generated from it.

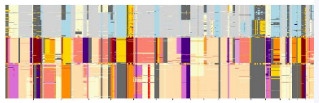
Phylogenies



State: tumor evolution tree
Measurement: DNA-seq

[Ivanovic & El-Kebir, RECOMB/Genome Res. 2023]

CNA profiles



State: copy-number profile
Measurement: read depth + BAF

[Ivanovic & El-Kebir, Genome Biol. 2025]

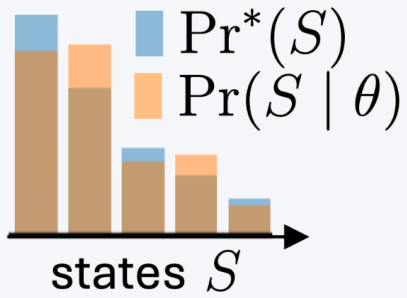
RNA isoforms

State: spliced transcript
Measurement: aligned short reads

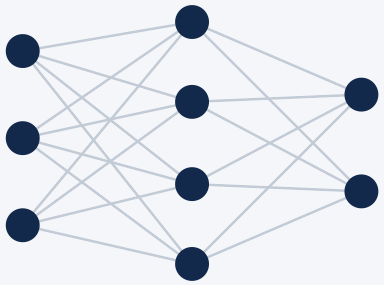
[Ivanovic et al., ICML 2026]

A learning and an inference problem

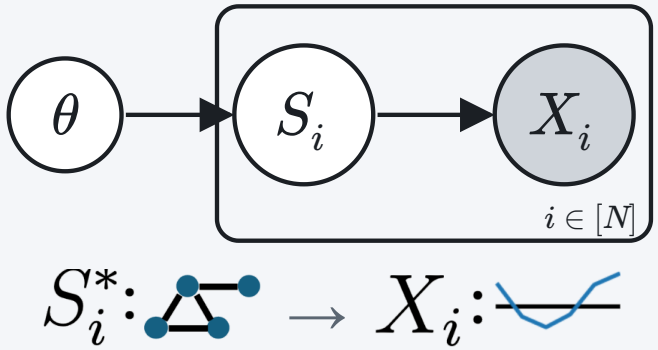
Approximate $\Pr^*(S)$ as $\Pr(S \mid \theta)$



Parameters θ: a linear map or neural network



Generative model with given $\Pr(X \mid S)$



Problem 1 (Learning). *Given (i)* $X_{1:N}$ *and (ii)* $\Pr(X \mid S)$*, find* θ *maximizing* $\Pr(X_{1:N} \mid \theta) = \prod_i \Pr(X_i \mid \theta)$*, where* $\Pr(X_i \mid \theta) = \sum_S \Pr(X_i \mid S)\ \Pr(S \mid \theta)$

Problem 2 (Inference). *Given (i)* $X_{1:N}$*, (ii)* $\Pr(X \mid S)$*, and (iii)* θ*, find* $\hat{S}_{1:N}$*, where* $\hat{S}_i = \arg\max_S \Pr(X_i \mid S)\ \Pr(S \mid \theta)$

Why the usual tools struggle

Approach	Problem
Expectation–Maximization	E-step expectation over $\mathcal{S}$ is **intractable** when $\mathcal{S}$ is combinatorial
Variational inference	maximizes an *ELBO* bound, not the likelihood — needs a **tractable posterior family** over combinatorial $\mathcal{S}$
Variational autoencoders	learn *artificial* continuous latents — **not** the mechanistic S you want
Local search ($\arg\max_S \Pr(X_i \mid S)$)	ignores the **shared** model $\Pr(S \mid \theta)$ across observations
Naive policy gradient	collapses to the single **highest-reward** state
GFlowNets	need **known terminal rewards**, not a likelihood to maximize

> **Gap:** none of these directly maximize $\Pr(X_{1:N} \mid \theta)$ over a *discrete, combinatorial* state space.

The GReinSS idea

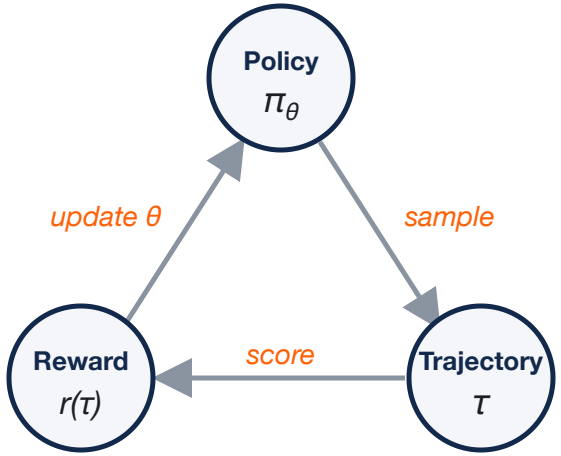
Build the discrete state step by step with a policy, and reward trajectories by their share of each observation's likelihood.

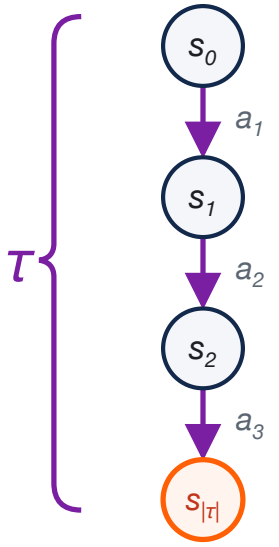
Generative Reinforcement Learning of Structured States

Primer on reinforcement learning (RL)

Key question: What actions should an agent take to maximize a reward signal?

Concept	In RL
State s_t	current situation
Action a_t	transitions $s_t \to s_{t+1}$
Policy π_θ	picks the next action
Trajectory τ	path $s_0 \to \cdots \to s_{\lvert\tau\rvert}$
Reward $r(\tau)$	scores terminal state
Objective	$\max_\theta \mathbb{E}_\tau[r(\tau)]$





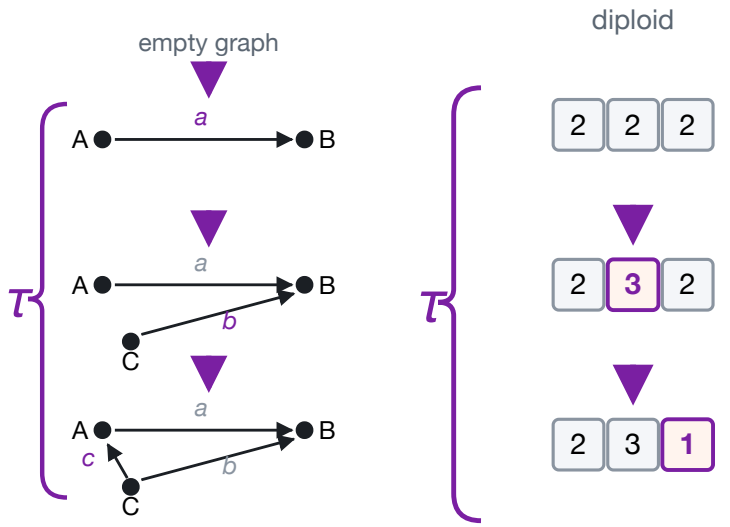
a trajectory τ

Policy gradient (REINFORCE): $\nabla_\theta \mathbb{E}_{\tau\sim\Pr(\tau|\theta)}[r(\tau)] = \mathbb{E}_\tau\left[r(\tau)\,\nabla_\theta \log \Pr(\tau \mid \theta)\right]$ –
gradient through $\log \Pr(\tau \mid \theta)$ *only, not the fixed reward* $r(\tau)$.

GReinSS: Generative Reinforcement Learning of Structured States

Policy gradient (REINFORCE): $\nabla_\theta \, \mathbb{E}_{\tau \sim \Pr(\tau|\theta)}[r(\tau)] = \mathbb{E}_\tau \left[r(\tau) \, \nabla_\theta \log \Pr(\tau \mid \theta) \right]$

Concept	In RL	In GReinSS
State s_t	current situation	*same as RL*
Action a_t	transitions $s_t \to s_{t+1}$	*same as RL*
Policy π_θ	picks the next action	*same as RL*
Trajectory τ	path $s_0 \to \cdots \to s_{\lvert\tau\rvert}$	*same, terminal* $S(\tau)$
Reward $r(\tau)$	scores terminal state	**use** $\Pr(X \mid S)$**???**
Objective	$\max_\theta \mathbb{E}_\tau[r(\tau)]$	$\max_\theta \log \Pr(X_{1:N} \mid \theta)$



Key question: How to set rewards $r(\tau)$ such that
$\nabla_\theta \, \mathbb{E}_{\tau \sim \Pr(\tau|\theta)}[r(\tau)] = \mathbb{E}_\tau \left[r(\tau) \, \nabla_\theta \log \Pr(\tau \mid \theta) \right] = \nabla_\theta \log \Pr(X_{1:N} \mid \theta)$?

Dynamically-changing rewards

Train with a policy gradient using the **dynamically rescaled reward** $r(\tau) = \sum_{i=1}^{N} \frac{\Pr(X_i \mid \tau)}{\Pr(X_i \mid \theta)}$.

$$\nabla_\theta \log \Pr(X_{1:N} \mid \theta) = \mathbb{E}_\tau\Big[r(\tau)\, \nabla_\theta \log \Pr(\tau \mid \theta)\Big]$$

what we optimize

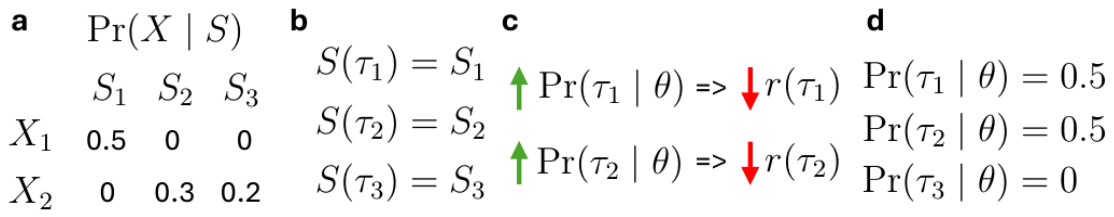
how we optimize it

Theorem 1 (Unbiased policy gradient). *With the dynamically changing reward* $r(\tau) = \sum_{i=1}^{N} \Pr(X_i \mid \tau) / \Pr(X_i \mid \theta)$, *the policy gradient* $\mathbb{E}_\tau\Big[r(\tau)\, \nabla_\theta \log \Pr(\tau \mid \theta)\Big]$ *is an unbiased estimator of* $\nabla_\theta \log \Pr(X_{1:N} \mid \theta)$, *the gradient of the log-likelihood objective.*

So standard policy-gradient ascent with this reward = **maximum-likelihood** learning of $\Pr(S \mid \theta)$.

Why the denominator? (intuition)

The rescaling rewards a trajectory for its **proportional contribution** to $\Pr(X_i \mid \theta)$ — not its raw probability. This makes the policy **spread** over the states the data needs, instead of collapsing to one.



Toy: 3 states, 2 observations.
$\Pr(X_1|S_1) = .5,\ \Pr(X_2|S_2) = .3,\ \Pr(X_2|S_3) = .2$

Naive PG → chases the max reward (τ_1):
$\Pr(\tau_1) = 1 \Rightarrow \Pr(X_2 \mid \theta) = 0$
$\Rightarrow \Pr(X_{1:N} \mid \theta) = \mathbf{0}$
✗

GReinSS → rewards balance out:
$\Pr(\tau_1) = \Pr(\tau_2) = 0.5,\ \Pr(\tau_3) = 0$
$\Rightarrow \Pr(X_{1:N} \mid \theta) = 0.0375$ ✓ *(optimal)*

The reward is *dynamic* — that's the whole trick

The denominator $\Pr(X_i \mid \theta)$ is **not known up front** and **changes every step** as θ updates — so we **re-estimate it by sampling** each iteration.

$$r(\tau) = \sum_{i=1}^{N} \frac{\Pr(X_i \mid \tau)}{\boxed{\Pr(X_i \mid \theta)}}$$

moves during training

$$\longrightarrow \quad \Pr(X_i \mid \theta) = \mathbb{E}_{\tau \sim \Pr(\tau \mid \theta)}\big[\Pr(X_i \mid \tau)\big]$$

estimated by sampling

> Recomputed after every update, the moving denominator continuously **rebalances** which states the policy must cover. This is the "*dynamic*" in **dynamic policy gradients**.

The training loop

Repeat until convergence:

1. **Sample** trajectories $\tau \sim \Pr(\tau \mid \theta)$
2. **Score** each with $\Pr(X_i \mid \tau)$
3. **Rewards** $r(\tau) = \sum_i \Pr(X_i \mid \tau) / \Pr(X_i \mid \theta)$
4. **Policy-gradient step** on θ
5. **Recompute** $\Pr(X_i \mid \theta) \rightarrow$ new rewards

You provide only two things:

- a way to **generate** S (grow it action-by-action)
- the likelihood $\Pr(X \mid S)$

Everything else is generic.

Mini-batching over observations keeps it unbiased and scalable (Thm.).

Off-policy learning (when on-policy is too slow)

If sampling $\Pr(\tau \mid \theta)$ rarely produces states that explain any X_i, learning stalls.

Sample where the data says to look — bias toward the Bayes posterior:

Theorem 2 (Optimal off-policy proposal). *The unbiased, variance-minimizing sampling proposal is*

$$q(\tau \mid X_{1:N}, \theta) = \tfrac{1}{N} \sum_{i=1}^{N} \Pr(\tau \mid X_i, \theta), \qquad \Pr(\tau \mid X_i, \theta) = \frac{\Pr(X_i \mid \tau)\, \Pr(\tau \mid \theta)}{\Pr(X_i \mid \theta)}.$$

Importance sampling keeps the gradient correct. *In cancer apps, a cheap heuristic (e.g. CNNaive in CNRein) seeds plausible states.*

The scoreboard — who actually maximizes the likelihood?

Method	Paradigm	Max. $\prod_i \Pr(X_i \mid \theta)$?
GReinSS	RL	✅ **Yes**
Naive policy gradient	RL	❌ No
GFlowNets	RL	❌ No
VAE / autoregressive / diffusion **+ EM**	EM	≈ approx.
Local search	per-observation	❌ No

> One green **Yes**. The RL cousins optimize the *wrong* target; EM methods only *approximate* it; local search ignores the **shared** model entirely.

→ NOTEBOOK · Demo 1: Set reconstruction

Recover binary **sets** from noisy real-valued measurements. *Trains live in ~10 s.*

Problem. $S_i^* \subseteq \mathcal{U}$, observe
$X_{i,j} \sim \mathcal{N}(1, \sigma^2)$ if $j \in S_i^*$, else $\mathcal{N}(0, \sigma^2)$.

You'll write:

```
def log_pr_x_given_g(state, obs):
    return -0.5*np.sum((obs-state)**2)/sigma**2
```

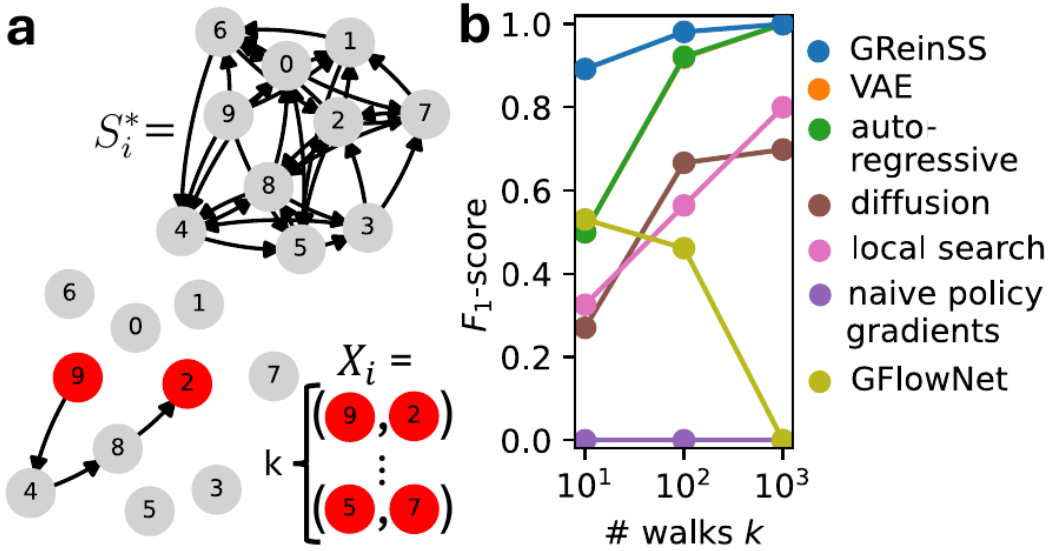
…then `simpleTrainModel(...)`.

Watch for:

- median $\log \Pr(X_i \mid \theta)$ climbing to ~0
- recovered sets vs. thresholding the noise
- where the **shared model fixes** noisy bits

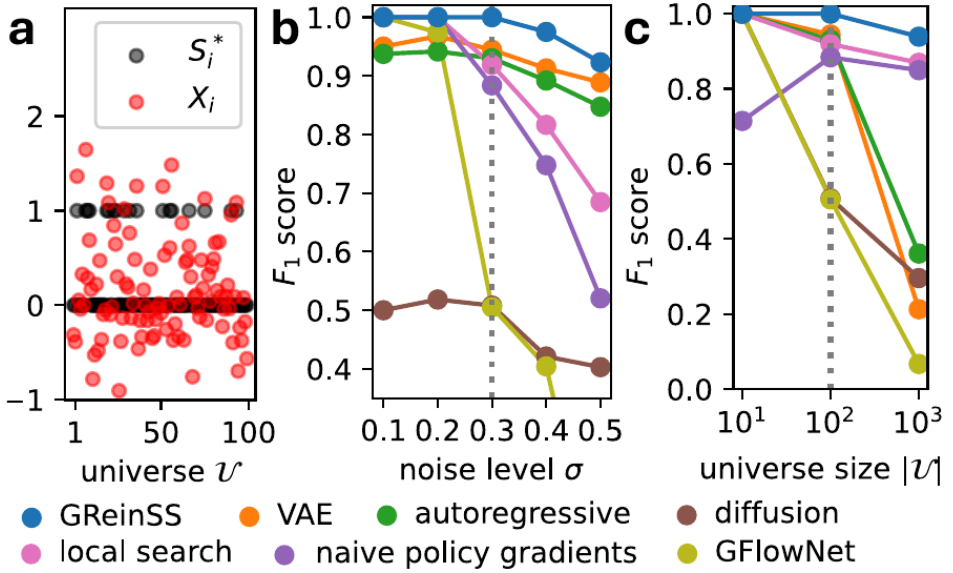
Results — simulations

Latent graphs from random-walk endpoints



$k = 10$ walks: GReinSS $F_1 = \mathbf{0.891}$; all baselines < 0.55

Latent sets from noisy measurements



$|\mathcal{U}| = 1000$: GReinSS $F_1 = \mathbf{0.938}$; GEM baselines < 0.4

Dynamic rewards are a **small** code change over naive PG — but decisive. Naive PG here predicts the **empty graph** ($F_1 = 0$).

→ NOTEBOOK · Demo 2: Graph inference (pre-trained)

Latent **directed graphs** from start/end points of k absorbing random walks.

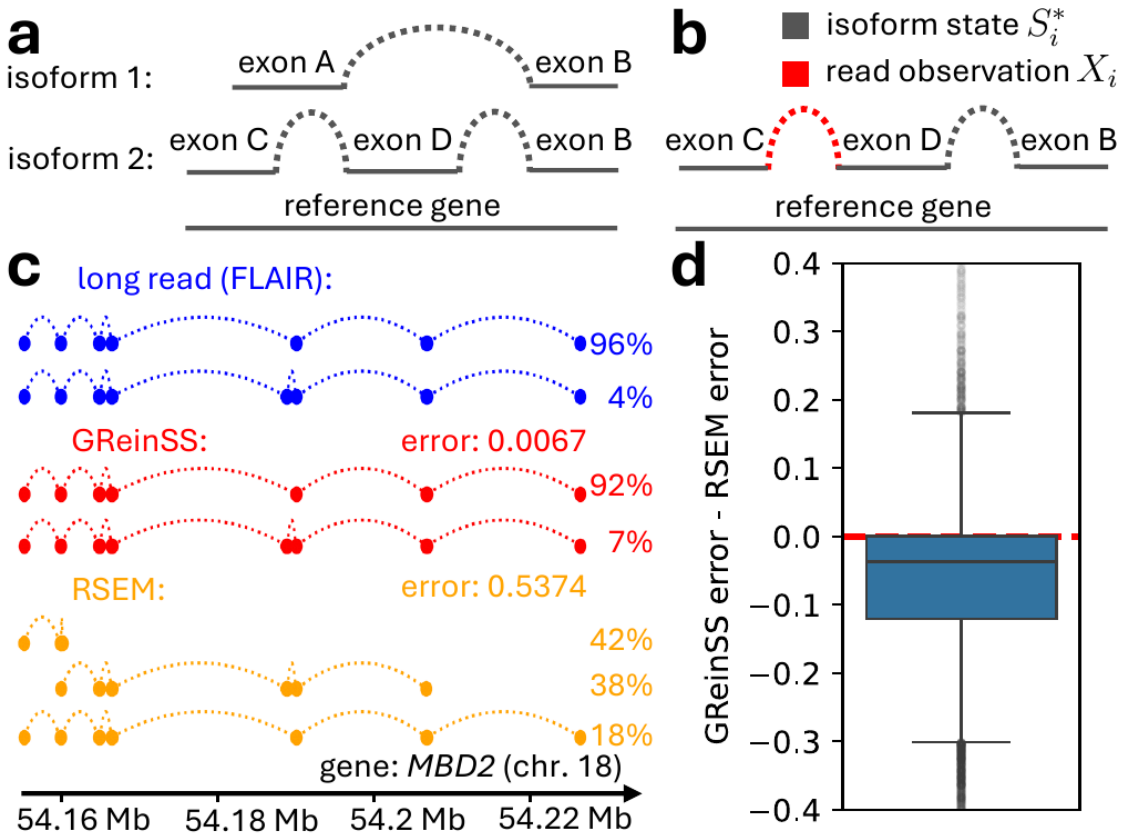
State = 90 possible directed edges (10 nodes). $\Pr(X \mid S)$ from the shifted-Laplacian $(L + I)^{-1}$ random-walk model.

We **load a pre-trained policy**, run inference, and score edge-recovery F_1 against ground-truth graphs.

Watch for:

- training likelihood curve (pre-computed)
- a reconstructed graph $\hat{S}_i$ vs. true S_i^*
- per-graph F_1 distribution

Application — RNA isoforms beat RSEM



State S = an isoform (chosen exon junctions) + sample + read position.
$\Pr(X \mid S) = 1$ iff read position & sample match — trivial forward model.

On **GTEx** (61 samples w/ matched long reads, 14,390 genes):

- GReinSS **beats RSEM by ≥0.05** on **46.6%** of genes; RSEM beats GReinSS on only **9.4%**
- *MBD2* example: GReinSS error **0.007** vs RSEM **0.537**

GReinSS already powers two cancer methods

CloMu

Tumor **phylogenies** of SNVs

States: mutation trees

Observations: noisy DNA-seq trees

Ivanovic & El-Kebir, RECOMB / Genome Res. 2023

CNRein

Copy-number evolution in single cells

States: sets of CNA events per cell

Observations: single-cell DNA-seq

Ivanovic & El-Kebir, Genome Biol. 2025

This paper **generalizes** the shared technique behind both into one framework for *any* discrete latent structure — with theory, off-policy theory, and baselines.

When should *you* reach for GReinSS?

Good fit ✓

- latent state is **discrete / combinatorial**
- you can **generate** it incrementally
- you know (or can compute) $\Pr(X \mid S)$
- observations are **indirect** & shared structure exists

Reach elsewhere ✗

- continuous latents → VAE / diffusion
- tractable exact E-step → classic EM
- rewards truly fixed & known → standard RL / GFlowNet

Recipe: ① write a generator for S · ② write $\Pr(X \mid S)$ · ③ `simpleTrainModel` · ④ `simpleInference`. *(optional)* add an off-policy proposal for hard instances.

Thank you — let's build

Code + notebook: `code/README.md`, `tutorial/GReinSS_demo.ipynb`
Recipe: generator for S + likelihood $\Pr(X \mid S)$ → train → infer

Questions? · melkebir@illinois.edu